

Interview Preparation: 50 Questions & Answers

Distributed Background Job Processing System

FastAPI + PostgreSQL + Redis Streams + SQLAlchemy 2.0 + Docker Compose

Covers: architecture, system design decisions, concurrency & failure scenarios, testing & unit tests, security, performance, monitoring.

Architecture & System Design

Q1. Walk me through the high-level architecture. Why these components?

FastAPI accepts submissions and writes the job row to PostgreSQL (source of truth), then enqueues a message to Redis Streams. Independent worker processes consume via a consumer group, execute handlers, and persist results back to Postgres. A single 'ticker' service handles time-based work: promoting scheduled/delayed jobs, reaping stale in-flight messages, and reconciling orphans. Postgres owns state; Redis only owns dispatch - so losing Redis never loses job state.

Q2. Why Redis Streams with a consumer group instead of a simple Redis list (LPUSH/BRPOP)?

A list pop is destructive: if the worker crashes after BRPOP, the message is gone. Streams with XREADGROUP move the message into a Pending Entries List (PEL) owned by that consumer, so an unacked message survives a crash and can be reclaimed with XAUTOCLAIM. That gives at-least-once delivery with ownership tracking for free.

Q3. Why is PostgreSQL the source of truth rather than Redis?

Redis is fast but its durability guarantees (RDB/AOF trade-offs) are weaker, and job state needs transactional updates, rich querying (list/filter endpoints), and long retention. Redis holds only transient dispatch signals; every state transition is committed in Postgres first. If Redis is wiped, jobs can be re-enqueued from Postgres via the reconciliation path.

Q4. What happens, step by step, when a job is submitted?

1) API validates the payload against the typed schema for the job type. 2) Idempotency key is checked - an existing job is returned instead of creating a duplicate. 3) The Job row is inserted (pending or scheduled) and committed. 4) The message is XADDED to the priority-appropriate stream and the row is flagged `is_synced_to_redis`. If the process dies between commit and XADD, the reconciler re-enqueues rows still marked unsynced after a grace period.

Q5. Why separate worker processes instead of async tasks or threads inside the API?

The assignment requires it, but the real reasons: process isolation (a segfaulting C extension or OOM kills one job, not the API), independent horizontal scaling (docker compose up --scale worker=N), and independent deploys/restarts. In-request processing also ties job latency to HTTP request lifecycles.

Q6. How do you guarantee a job is picked up by only one worker?

Two layers. Redis Streams assigns each message to exactly one consumer in the group (PEL ownership). But because reclaim/redelivery can happen, the worker also does an atomic claim in Postgres: `UPDATE ... SET status='processing' WHERE id=? AND status IN ('pending','scheduled')`. If zero rows update, the message is a stale redelivery and is discarded. The database claim is the real mutual-exclusion guarantee; the stream is just efficient dispatch.

Q7. What is the ticker service and why is it a single instance?

It runs the periodic loops: `promote_due` (move due scheduled jobs from the delayed ZSET into streams), the reaper (`XAUTOCLAIM` stale PEL entries), and `reconcile_orphans` (re-enqueue rows never synced to Redis). It runs as exactly one replica because concurrent reapers without leader election would double-process; its

workload is light and does not need to scale, only workers do.

Q8. How are scheduled (future) jobs implemented?

A job with a future run time is stored as 'scheduled' in Postgres and added to a Redis sorted set (jobs:delayed) with the due timestamp as score. The ticker's promote_due polls the ZSET, and for due members flips the row to pending and XADDs to the correct priority stream. A drain-until-not-full loop with pipelined XADD, multi-member ZREM, and bulk Postgres updates avoids artificial lag when thousands become due at once.

Q9. How does priority work, and why three streams instead of a sorted set?

Three streams - high/normal/low - and workers probe them non-blocking in strict order, falling back to a blocking read across all three only when all are empty. A ZSET would give ordering but a pop is just a pop: no PEL, no XACK, no XAUTOCLAIM, so all the delivery-tracking and crash-recovery machinery would have to be hand-rolled. Per-priority streams reuse the consumer-group machinery as-is.

Q10. If Redis goes down entirely, what happens?

Submissions still commit to Postgres (rows stay is_synced_to_redis=False); enqueueing fails but no state is lost. Workers block on reads and retry. When Redis returns, the reconciler re-enqueues the unsynced rows. Completed/failed history is untouched since it lives in Postgres.

Concurrency & Failure Scenarios

Q11. A worker crashes mid-job. Walk me through recovery exactly.

The message stays unacked in the PEL and the row stays 'processing' - safe, because the claim-guard prevents double-claim. Once the PEL entry is idle past visibility_timeout_s (60s), the ticker reaper XAUTOCLAIMs it and routes it through the same schedule_retry_or_fail path as an ordinary handler failure: re-enqueue with backoff, or terminal 'failed' at max_attempts. If the crash happened after DB commit but before XADD, there is no message to reclaim - reconcile_orphans catches the unsynced row instead.

Q12. How do you handle a handler that hangs but whose process is still alive?

Each handler runs in a single-use thread bounded by job_handler_timeout_s (45s). Python cannot kill a thread, so on timeout the worker abandons it, raises HandlerTimeout into the retry path immediately, and then recycles its own process (max_handler_timeouts_before_recycle, default 1) so the zombie thread cannot leak resources. The config validates handler timeout < visibility timeout, so the in-process layer always wins the race against the reaper.

Q13. Why is ack (XACK) issued after the Postgres commit and not before?

Ack-after-commit means a crash between commit and ack leaves an unacked message that gets redelivered - and the claim-guard then discards it as already-claimed. Ack-before-commit would be worse: a crash after ack but before commit silently loses the state transition with no redelivery to recover it. The chosen order trades harmless redelivery for zero lost updates.

Q14. How does retry with backoff work?

Every failure - handler exception, handler timeout, or reaper-discovered crash - funnels into one shared schedule_retry_or_fail path. backoff_delay looks up retry_backoff_schedule=[0,30,120]s by attempt: first retry immediate (straight back to pending and re-enqueued), later ones go to 'scheduled' via the delayed ZSET. At attempts >= max_attempts (default 4) the job is marked failed permanently.

Q15. How is cancellation implemented, and what are its limits?

Pending/scheduled jobs are cancelled by an atomic status flip - if the worker's claim-guard then finds the row cancelled, it never runs. For in-flight jobs cancellation is cooperative: the handler must poll a cancellation flag, and only the batch handler does. You cannot safely kill arbitrary Python threads, so pre-emptive cancellation was ruled out.

Q16. What is the split-brain window in your design and how big is it?

Between handler completion and the Postgres commit, a slow worker that exceeds the visibility timeout could be reaped while still alive, and the job re-run. Mitigations: visibility timeout (60s) is deliberately above the handler timeout (45s) so the in-process timeout fires first, and the claim-guard plus idempotent handlers bound the damage to duplicate side effects, not corrupted state.

Q17. What if the same idempotency key arrives concurrently from two API requests?

The idempotency key has a unique constraint in Postgres, so one insert wins and the other hits a violation, which is caught and answered with the existing job. The database, not application-level checking, is what makes the check race-free.

Q18. What is a poison message and how does your system contain it?

A job that crashes its handler on every attempt. Attempt counting caps it at `max_attempts`, after which it is marked failed permanently with the error stored - the DB itself acts as the dead-letter record, queryable via the list endpoint with `status=failed`. A dedicated DLQ stream would be the next step for automated triage.

Q19. Graceful shutdown - how does a worker stop without losing its current job?

On `SIGTERM` the worker stops reading new messages and finishes the in-flight job through commit and `XACK` before exiting. If it is killed hard anyway, the normal crash-recovery path (`PEL` + reaper) takes over - graceful shutdown is an optimization, not a correctness requirement.

Testing & Unit Tests

Q20. How is the test suite organized and why?

`tests/unit` for pure logic with no external services - backoff math, timeout wrapper, payload/schema validation, handlers with fakes, config validation, cursor encoding, logging/observability. `tests/integration` for behavior that only exists with real Postgres and Redis - queue pickup, retry flow, reaper, delayed promotion, idempotency through the API, cancellation, batch progress, migrations. The split keeps the fast feedback loop fast while still exercising the real concurrency machinery.

Q21. Which of your tests would you call the six 'meaningful' required ones?

Submission and retrieval (`test_api`), completion flow end-to-end (`test_worker`), failure-and-retry with backoff (`test_retry`), cancellation of pending/in-flight (`test_cancel`), idempotency under duplicate submission (`test_idempotency_api`), and priority ordering - high drained before normal/low (`test_queue`). Each asserts observable state transitions, not implementation details.

Q22. How do you test worker crash recovery without actually crashing a process?

`test_reaper` simulates it at the protocol level: read a message with `XREADGROUP` but never `XACK`, advance/exceed the idle threshold, then run the reaper tick and assert the job was reclaimed and routed through retry with its attempt count incremented. Killing real processes would be slow and flaky; the `PEL`-idle state is the actual trigger, so producing that state directly tests the same code path.

Q23. How do you make time-dependent tests (backoff, delayed jobs, visibility timeout) deterministic?

Never sleep for real durations. Backoff is a pure function tested directly with attempt numbers. Delayed-job tests insert `ZSET` members with past-due scores and invoke `promote_due` directly. Reaper tests use the min-idle-time parameter of `XAUTOCLAIM` with tiny thresholds. Injecting or manufacturing time beats waiting for it.

Q24. How do you test the 80%-failure webhook handler deterministically?

Randomness is injected, not global: seed or stub the random source so the unit test forces both the success and failure branch explicitly. The retry integration test then uses a handler forced to fail `N` times before succeeding, so retry counting is asserted exactly rather than statistically.

Q25. What do integration tests use for Redis and Postgres, and why not fakes?

Real services from docker compose (fakeredis was deliberately removed - see git history). The system's core guarantees live in Redis-specific semantics: PEL behavior, XAUTOCLAIM idle rules, consumer-group ownership, atomicity of UPDATE...WHERE. Fakes diverge from those semantics exactly where the bugs are; a fake that passes proves little about the property being tested.

Q26. How would you test the claim-guard actually prevents double execution?

Simulate the race directly: claim the job once (status flips to processing), then attempt the second claim and assert it affects zero rows and the worker discards the message. Optionally, two concurrent claim calls in threads against real Postgres asserting exactly one winner - the atomicity comes from the database, so the test must hit real Postgres.

Q27. A test passes locally but fails in CI intermittently. Your process?

Treat it as a real bug until proven otherwise - flaky tests in a queue system usually mean a real race. Reproduce with the same seed/parallelism, look for timing assumptions (sleeps, ordering assumptions on stream reads), and fix by making the test event-driven (poll for the state transition with a deadline) rather than sleep-based.

Security

Q28. What input validation exists at the API boundary?

Every job type has a typed Pydantic payload schema (schemas/payloads.py) - unknown job types are rejected, field types and constraints are enforced, and the payload stored in Postgres is the validated, serialized model, not raw client JSON. List-endpoint filters and pagination cursors are also validated (cursor encoding has its own unit tests).

Q29. The payload is stored as JSON and later executed by handlers. Injection risks?

Handlers never eval or template payload content into anything executable; payloads are data consumed by mock handlers. SQL access goes exclusively through SQLAlchemy bound parameters - no string-built SQL anywhere - so payload content cannot become SQL. The job type maps to a handler via an explicit registry dict, never by dynamic import or getattr from user input.

Q30. How do you prevent one tenant/client from reading another's jobs?

Currently there is no authentication - acceptable for the assignment scope and documented. The design point I would make: job IDs are UUIDs (non-enumerable), and the first production step is authenticating callers and scoping every query by an owner column; the repository layer centralizes queries so that is one choke point.

Q31. What would you add before exposing this API publicly?

AuthN/AuthZ (API keys or OAuth client credentials), per-caller rate limiting on submission, payload size limits, TLS termination, restricting CORS, and audit logging of cancel/retry actions. Also queue-depth quotas per caller so one client cannot starve the system - a security-adjacent availability concern.

Q32. Are there secrets in the repo? How are credentials handled?

Connection strings come from environment variables via the Settings object; docker-compose supplies dev-only credentials. The git history shows a 'fixed security' commit and .gitignore updates for exactly this hygiene. In production: a secrets manager, least-privilege DB users (workers do not need DDL), and Redis AUTH plus network isolation.

Q33. Could a malicious payload DoS a worker?

A payload engineered to make a handler hang is bounded by the 45s handler timeout plus process recycle; one that always crashes is bounded by max_attempts. Oversized payloads are the remaining gap - I would enforce a request body limit at the API. The one-job-per-process model means the blast radius of any single job is one container.

Performance & Scalability

Q34. Where is the throughput bottleneck in this design?

Per-worker throughput is bounded by one-job-at-a-time, so aggregate throughput scales with replica count until Postgres becomes the bottleneck - every job costs a claim UPDATE, a completion UPDATE, and reads. Redis Streams will handle far more ops/sec than Postgres will absorb writes. First mitigations: batch/reduce non-essential writes, connection pooling (already via SQLAlchemy), then read replicas for query endpoints.

Q35. 10,000 scheduled jobs become due in the same second. What happens?

This exact case drove the ticker's drain-until-not-full loop: instead of promoting a fixed page per tick (which would add ~100s of artificial lag), it keeps draining while there is due work, and pipelines the Redis XADDs, uses multi-member ZREM, and a bulk Postgres UPDATE to amortize round trips. Workers then chew through the burst in parallel.

Q36. How does the worker read loop avoid busy-polling across three priority streams?

read_priority probes high, normal, low non-blocking and returns on the first hit; only when all three are empty does it issue a single blocking XREADGROUP across all three. So an idle system parks in one blocking call (no CPU burn), and a busy system pays at most two extra cheap probes per message.

Q37. What is the latency cost of your crash-recovery design?

A crashed worker's job waits up to visibility_timeout_s (60s) before the reaper reclaims it - that is the recovery latency floor. It is a deliberate trade against false positives (reaping a live-but-slow worker). The heartbeat design in DECISIONS #5 would cut detection latency to the heartbeat interval without raising false-positive risk.

Q38. How would you scale this 10x? 100x?

10x: more worker replicas, tune Postgres pool sizes, indexes already on status/priority - mostly a knob-turning exercise. 100x: Postgres write volume becomes the wall - partition the jobs table by time, archive terminal jobs out of the hot set, move job logs to append-optimized storage, possibly shard streams per job type. The ticker stays single-instance far longer than intuition suggests because its work is batched.

Q39. Which indexes exist and why?

Status (list filtering and the reconciler's unsynced scan), priority (filtering), idempotency key (unique - both correctness and lookup speed), and scheduled-time for promotion queries. Composite candidates like (status, priority) would come from EXPLAIN on the real list workload rather than speculation.

Monitoring & Observability

Q40. What does the health endpoint report and why those stats?

Liveness of dependencies (Postgres and Redis reachable) plus queue statistics: per-status job counts, stream depths per priority, delayed-set size, and PEL size. Those specific numbers are chosen because each maps to a failure mode: growing PEL means dead/stuck workers, growing delayed set means the ticker is behind, growing pending with idle workers means dispatch is broken.

Q41. Describe your logging approach.

structlog structured JSON everywhere - the project rule is no print statements, only structured logging with job context. Worker context binds job_id, job_type, attempt, and consumer name onto every log line for the duration of a job, so one job's full lifecycle across API, worker, and ticker is a single filter query in any log aggregator.

Q42. What metrics would you export first if you added Prometheus?

Counters: jobs submitted/completed/failed/retried/reaped by type and priority. Histograms: queue wait time (enqueue to claim) and execution time by type. Gauges: stream depth, PEL size, delayed-set size, worker count. The single most valuable alert is queue wait time p99 - it captures almost every failure mode (worker death,

backlog, ticker lag) in one signal.

Q43. How would you detect that the ticker died?

Its absence is visible in the metrics it stops moving: delayed-set size grows past due, PEL max idle age climbs unbounded. Directly: the ticker should heartbeat (touch a TTL key or emit a tick metric) and an alert fires when the heartbeat goes stale. Since it is single-instance, this is the most important liveness alert in the system.

Q44. A user reports their job has been 'processing' for an hour. How do you investigate?

Query the job row for attempts, started_at, and consumer name; grep structured logs by job_id for the last event. Check whether its stream message is still in the PEL and its idle time. Given the 45s handler timeout and 60s visibility timeout, an hour-long 'processing' state implies the reaper is not running - check ticker liveness first - or the job's message was lost, which reconcile_orphans should have caught; the is_synced_to_redis flag on the row distinguishes those.

Q45. How do you trace one job across API, ticker, and worker?

job_id is the correlation key bound into every structured log line by all three services. With OpenTelemetry I would add a trace/span per lifecycle stage and propagate context through the stream message fields, but job_id-keyed structured logs already give the full timeline.

Design Decisions & Trade-offs (DECISIONS.md deep-dive)

Q46. Which decision would you defend hardest under challenge?

Ack-after-commit with the atomic DB claim-guard. It is the keystone: it made the reaper safe to add (Decision #2), made stream redelivery harmless (Decision #1), and let retries share one path (Decision #4). Every other component leans on 'redelivery is safe', and that property comes from this one ordering choice.

Q47. Which decision is most likely to be wrong at scale?

Strict priority with no fairness, and the global retry policy. A sustained high-priority flood starves everything else, and one backoff schedule for a cheap webhook and an expensive report is crude. Both have documented, non-invasive escape hatches: per-worker stream restriction, and a per-submission max_attempts/backoff override field.

Q48. If you rebuilt this with zero constraints, what changes?

Per DECISIONS #5: heartbeat-based liveness instead of fixed visibility timeouts. Beyond that: per-job-type retry policies, a real DLQ stream, and possibly Postgres-native queueing (SELECT ... FOR UPDATE SKIP LOCKED) to remove Redis entirely at moderate scale - fewer moving parts, at the cost of Postgres doing double duty.

Q49. Why not just use Celery/RQ/Sidekiq-style off-the-shelf tooling?

The assessment evaluates understanding of the concurrency problems those tools hide. But it is also a defensible production stance: the requirements (priority streams, DB-as-truth claim-guard, custom reaper semantics, per-row idempotency) sit exactly in the zone where fighting a framework's opinions costs more than owning ~2k lines whose failure modes you fully understand.

Q50. What did AI tooling get wrong during this build, and how did you catch it?

The classic traps documented in AI_USAGE: check-then-act race patterns (checking status in Python before updating, instead of an atomic conditional UPDATE), ack-before-commit orderings that lose updates, and fakeredis-based tests that pass while real Redis semantics differ - fakeredis was removed from the repo for exactly that reason. Each was caught by asking 'what happens if the process dies between these two lines' - the question AI-generated code most consistently fails.